

동양생명

생성형AI 코드에이전트 시스템 구축

I

제안 개요

1.1 제안 배경 — 금융·보험 개발 표준으로 부상한 AI 코드 에이전트

글로벌 금융·보험사는 AI 코드 에이전트를 단순 자동완성이 아니라 표준 준수, 코드 이해, 반복 업무 자동화를 검증하는 개발 실행 환경으로 활용하고 있습니다

◆ 금융권 AI 개발 전환

AI 코드 에이전트의 부상

1 자동완성에서 자율 에이전트로

코드 AI는 자동완성·대화형 보조를 지나, 계획을 세우고 승인을 받아 스스로 실행하는 자율 코드 에이전트로 진화하고 있습니다

2 속도가 아니라 '이해'

스니펫 생성이 아니라 대규모·레거시 코드베이스를 이해하고 탐색해 작업을 수행합니다 — 핵심은 타이핑 속도가 아니라 코드의 '이해'입니다

3 엔터프라이즈 도입 가속

개발 방식 자체가 바뀌고 있고, 기업의 코드 에이전트 도입도 빠르게 늘고 있습니다

금융·보험업 공개 도입 사례

1 GNP Seguros — 멕시코 보험사 공개 사례

대규모 개발 조직에서 실제 코드·CI/CD 기반 자율 SW 개발 파일럿을 수행
-> 대규모 보험사도 전면 도입보다 실제 코드 기반 파일럿과 보안·기술 표준을 입력값으로 삼아 단계적으로 확산하고 있습니다.

(출처: 공개 발표자료)

2 Munich Re — 세계 최대 재보험사

상품 설계·구현에 AI 어시스턴트를 실전 투입
-> AI가 보험 개발 사이클 안으로 들어왔습니다

3 국내 금융권

KB·하나·우리·롯데손보 등은 내부망 기반 생성형 AI 플랫폼을 구축 중
-> 금융권 AI 도입은 보안·규제 대응을 위해 전용 AI 환경으로 확산되고 있습니다

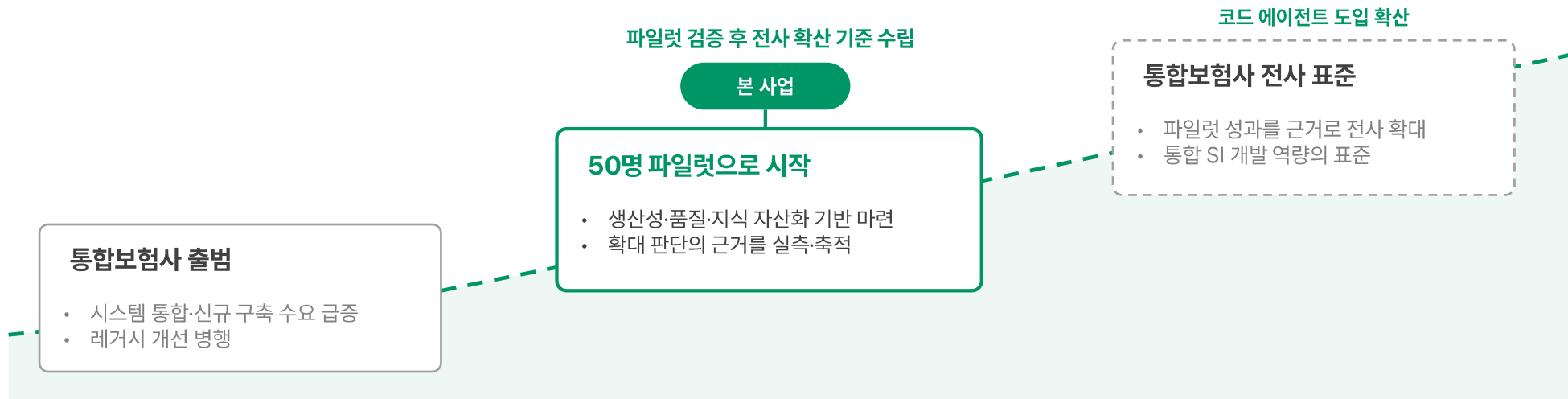
결론

AI 코드 에이전트는 금융·보험 개발의 새로운 표준이 되고 있으며, 동양생명도 안전한 온프레미스 기반으로 조기 내재화해야 합니다

1.1 제안 배경 — 통합보험사 출범에 따른 개발 표준화 과제

통합보험사 출범 이후 개발 품질 편차, 지식 개인화, 반복 업무 부담을 줄이기 위해 50명 파일럿에서 표준 적용 방식과 확산 기준을 먼저 검증해야 합니다

◆ 통합보험사 개발 표준화



개발 현장의 3가지 문제

인력별 편차

개인마다 코드의 품질·개발의 속도가 다릅니다

조직 전체 품질이 균질하지 않습니다

지식의 속인화

지식이 개인 머릿속에 있습니다

사람이 떠나면 지식과 노하우도 함께 떠납니다

반복 업무 부담

이해·탐색·문서화에 많은 시간을 소모합니다

정작 설계·판단에 쓸 시간이 부족합니다

결론

본 사업은 단순 코딩 도구가 아니라, 동양생명의 코드와 표준을 이해하는 전용 코드 에이전트 구축 사업입니다

본 사업의 핵심은 코딩 도구 도입이 아니라, 동양생명의 코드·표준·업무 맥락을 DeepCode가 참조하고 승인 가능한 방식으로 실행하는 개발 기반 구축입니다

**결론**

코드 에이전트는 지식DB와 결합될 때 동양생명의 코드를 이해하고, 안전하게 실행하는 전사 개발 자산이 됩니다

1.3 핵심 콘셉트 — 검증된 에이전트 경험을 폐쇄망 온프레미스로 구현

DeepCode는 검증된 에이전트형 개발 UX를 동양생명 폐쇄망 기준으로 재구현하고, 소스·지식·실행 기록이 사내망 밖으로 나가지 않는 구조를 제공합니다

◆ 핵심 구현 축

코드 에이전트 축 · DeepCode

시장 검증 UX 패턴을 폐쇄망 요건에 맞게 자체 구현

계획 → 승인 → 실행

HIL 승인 게이트 내재화 — 실행 전 사람이 검토·승인

반복 업무 즉시 실행

검증된 스킬 카탈로그로 재현·자동화

사내 시스템·사내 승인 모델 또는
온프레미스 모델 게이트웨이 연계
MCP 표준 확장으로 폐쇄망 안전 연결

사내 지식DB 축

레거시를 위키로 만들어 에이전트가 직접 탐색하는 사내 지식DB

레거시를 위키(SoT)로 정리

영업지원시스템 코드·문서를 자동 정리

에이전트가 직접 탐색

사람이 코드 읽는 순서로 온디맨드 탐색

원문 기반 참조 — 출처 추적 가능한 이해

조각내지 않아 정확하고 항상 최신

DeepCode는 개발자의 기존 업무 흐름은 유지하면서, 승인·기록·지식 참조를 결합한 온프레미스 코드 에이전트 환경을 제공합니다

1.4 제안 범위 — 기존 개발환경, DeepCode, 위키형 코드지식DB 연결

개발자는 기존 IDE·CLI·데스크톱 환경을 유지하고, DeepCode와 사내 MCP/npm 배포 체계를 통해 위키형 코드지식DB를 활용합니다

◆ 개발환경·지식DB 연결

본 제안의 강점은 익숙한 UI 제공, 손쉬운 연결, 그리고 높은 성능과 운영성입니다



소유권 범례

- 산출물 = **동양생명 귀속**
- 솔루션 원제품 = **라이선스 제공**

■ 다른 선택 — 요건은 충족하되 방식이 다른 3건

환경 그대로 : 쓰던 IDE·터미널 그대로라 재교육·환경 이전이 없습니다

손쉬운 연결 : npm 레지스트리 플러그인 설치 한 번으로 전사 확산이 쉽습니다

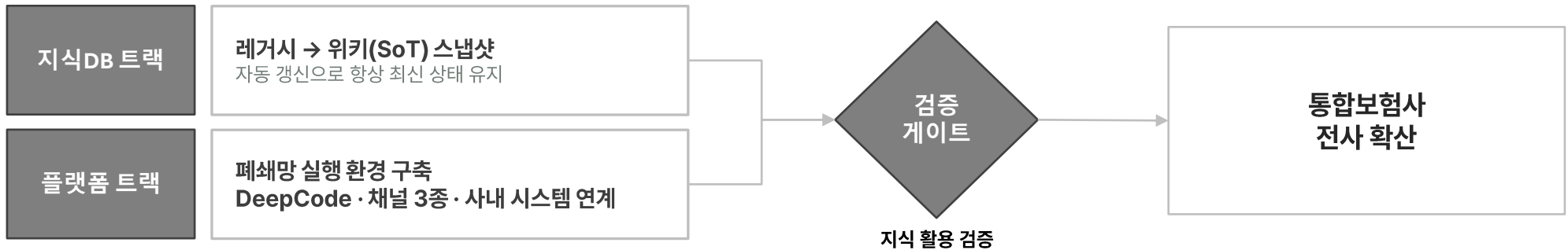
벡터 대신 위키 인덱싱·벡터DB 없이 OKF를 활용 — 현 LLM의 성능 잠재력 극대화 + 운영성 탁월

지식DB 구축과 플랫폼 적용을 병렬로 추진하되, 생산성·표준 준수·보안·사용자 채택 기준을 통과한 사용 방식만 확산 후보로 전환합니다



병렬 실행 구조

서로 독립적인 두 축을 병렬로 세워 → 통합 게이트에서 결합·검증 → 전사 확산



추진 원칙

검증 우선 파일럿으로 증명한 뒤 확대	병렬 · 격리 두 트랙을 분리해 위험 차단	표준 내재화 검증된 표준을 전사 자산으로
-------------------------	----------------------------	---------------------------

핵심 리스크

규제 준거 ✓ 금융위 금융분야 AI 가이드라인 준수	소스 · 보안 ✓ 소스 반출 불가 · 서버에서 권한 집행	현업 채택 ✓ 쓰던 IDE · CLI 그대로, 재교육 최소화
---------------------------------	------------------------------------	--------------------------------------

1.6 기대효과 — 반복 작업 중심 개발에서 표준 기반 개발 체계로 전환

DeepCode 도입 후 개발 방식은 수작업 작성·사후 발견·문서 노후화 중심에서 표준 기반 초안 생성, 실행 시점 검사, 위키형 지식 최신화, 승인 이력 축적이 연결되는 방식으로 전환됩니다

◆ 기대효과

AS-IS — 일반적 개발 관행

VS

TO-BE — 코드 에이전트 도입 후

1 수작업 작성

- 유사 CRUD·화면 코드를 건별로 수작업 작성
- 개발자 숙련도에 따라 품질 편차 발생

2 사후 발견

- 표준 위반을 리뷰·테스트 단계에서 발견
- 수정 비용이 공정 후반으로 갈수록 증가

3 문서 노후화

- 코드 변경 후 문서 갱신이 지연·노후화
- 레거시 지식이 담당자 개인에게 종속

4 쓸수록 정체

- 도입 시점 성능에 고정
- 사용 기록이 남지 않아 개선으로 이어지지 않음

1 표준 기반 초안 자동 생성

- 테이블 정의만 입력하면 표준 구조의 소스를 한 번에 생성
- 표준 아키텍처에 맞춰 CRUD·화면을 일관되게 생성합니다

2 실행 전 표준·보안 규칙 확인

- 표준·명명 규칙·보험 용어를 코드 생성 시점에 검사
- 문제를 초기에 걸러내 수정 비용을 줄입니다

3 위키형 코드지식DB 화

- 코드 수정과 문서 갱신을 한 사이클로 처리
- 문서가 항상 최신 코드와 같은 상태로 유지됩니다

4 승인·실행 이력이 개선 근거로 축적

- 실행 기록(Trace)이 데이터로 축적되어 Agent를 사용할 수록 강화학습을 통한 Data flywheel의 기반이 됩니다



서비스 구축

지식DB 구축과 솔루션 도입을 분리해 병렬로 진행하고, 핵심 의사결정 지점에서만 통합합니다

◆ 수행 구조

2-트랙 병렬 실행 로드맵 — 통합 게이트

① 지식 트랙 — 지식DB · 위키 구축

주도: 브레인크루(주사업자) DeepCode 기술 구축 · 딜로이트 PM/QA/리스크 및 지식DB 설계·개발 참여

진단·인터뷰

- 원천 접근(W1~2) · 인벤토리 · 표준 역도출
- 1개월차 · 진단 완료 게이트

프로파일·생성

- 지식 구조 프로파일 · 파이프라인 · 초기 번들
- 2개월차 · 파일럿 읽기 게이트

검수·갱신

- 검수 요청 승격 · 갱신 필요 순환 · 자동 코드리뷰
- 3개월차 · 쓰기 개방 게이트

② 플랫폼 트랙 — 솔루션 도입 · 연계 · 인프라

서비스 구축 수행 구조

지식 도구 계약·서버

- 도구 계약 확정 → 정책 서버 구축
- 정책 게이트 시범 가동

DeepCode 통합 · 에이전트 접점 검증 · 코드리뷰 루프 · 주입 테스트

인프라·배포

- 구성 명세 · 부하 · 백업
- 전환 계획 · DB 표준

③ 통합 게이트 — 두 트랙의 접점 검증

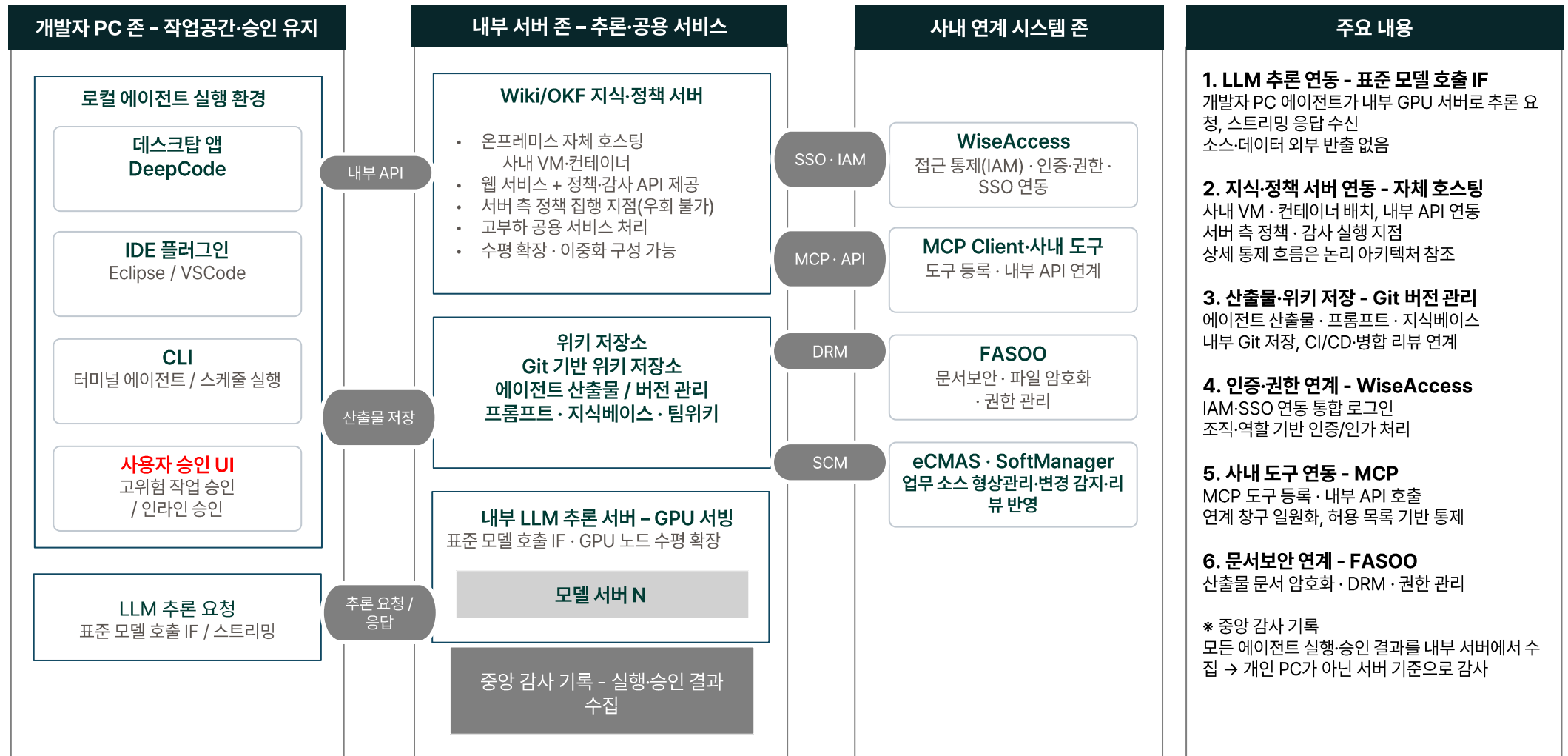
에이전트가 사내 위키를 실제로 탐색·활용하는 시나리오 검증

핵심 경로 — 원천 접근(W2 내) 지연 = 전체 지연 — 1주 초과 시 운영위 즉시 상신

소스와 데이터는 동양생명 내부망에 남고, 고부하 모델 추론과 공용 서비스는 내부 서버에서 처리합니다

◆ 구성도

동양생명 On-Premise Zone — 3존 아키텍처



화면, 실행, 지식, 감사의 4계층을 분리하고 승인·차단·기록 기능을 구조 안에 넣어 사용자가 임의로 우회하지 못하게 합니다

◆ 구성도

논리 아키텍처 — 4계층 구조



위험한 작업은 실행 전에 멈춰 승인을 받고, 승인되면 중단된 지점부터 이어서 실행합니다

◆ 흐름도



실행 모드 — 계획 / 실행

계획 모드 (읽기 전용)

- 조사와 계획만 수행 — 파일 쓰기 · 셀 · 설정 변경 · 외부 도구 위임 제외
- 변경 작업 경로를 구조적으로 차단

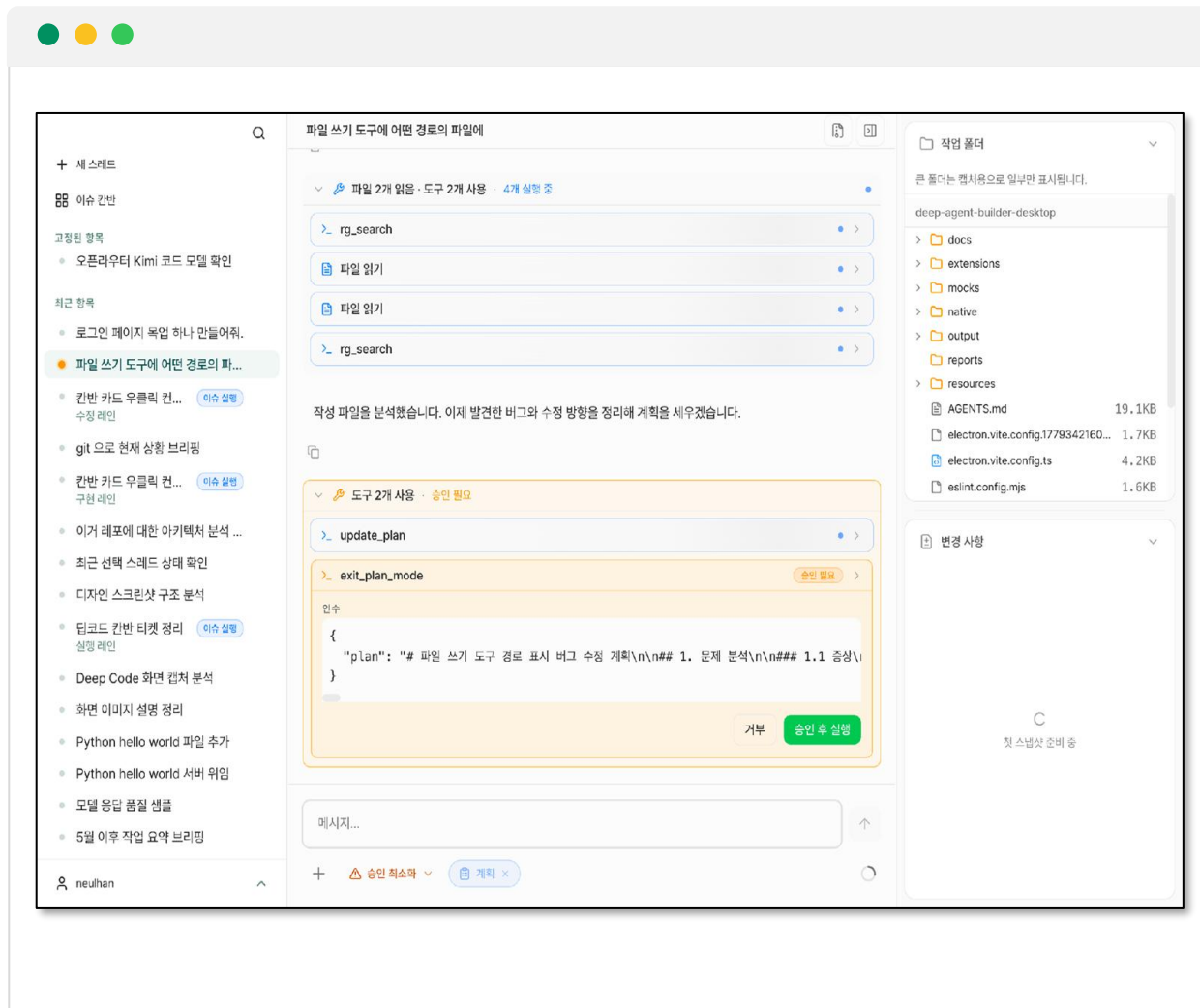
실행 모드 (기본)

- 파일 수정 · 셀 실행 · 도구 사용으로 실제 구현하는 기본 모드
- 위험 작업은 승인 게이트에서 정지

중단 · 재개 — 승인 대기로 멈춘 작업은 승인 시, 멈춘 지점부터 그대로 이어서 실행됩니다

승인 게이트와 계획 모드는 이미 동작하는 제품 화면으로 실증됩니다 — 위험 도구 호출은 '승인 필요'로 정지하고, 승인 후 실행을 선택하면 이어집니다

◆ 승인 게이트 화면



1 실행 전 정지

- 계획 승인 도구 호출이 '승인 필요'로 정지(중단)
- 인수(계획 전문)를 펼쳐 검토 가능

2 거부·승인 선택

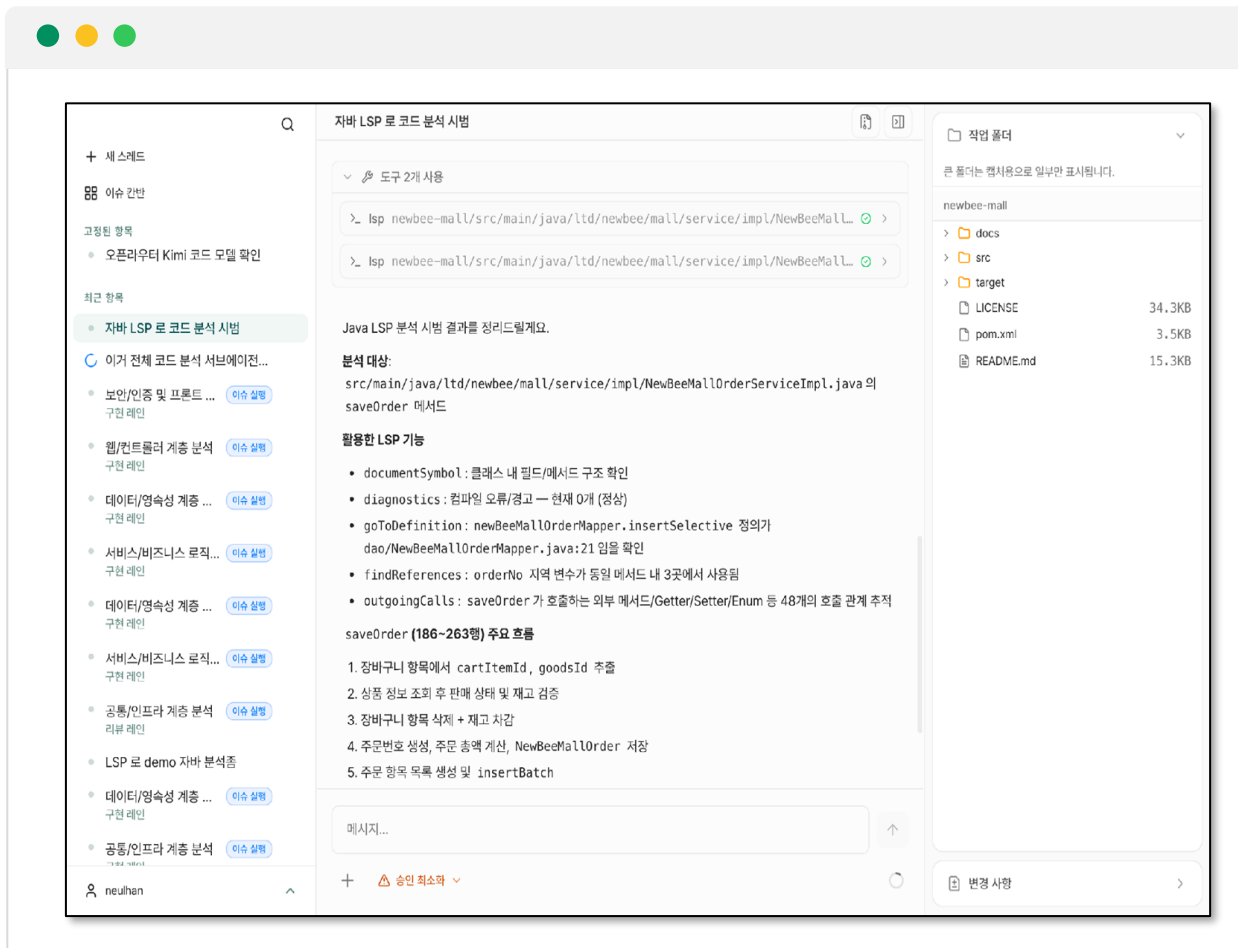
- '승인 후 실행' 선택 시 멈춘 지점부터 재개(재개)
- 거부 시 실행 중단

3 실행 내역 표시

- 계획 갱신 · 코드 검색 · 파일 읽기 진행 상태 표시
- 하단 작업 계획 배지 · 승인 최소화 옵션

정의와 참조를 언어 서버 기반 분석으로 추적해 오류를 고치고, 변경 영향은 작업 시점에 코드를 직접 확인해 산출합니다

◆ 코드 분석 화면



1 정의·참조 추적

- 정의 이동 — 호출부에서 정의 위치로 즉시 이동
- 참조 검색 — 정확한 참조 추적으로 수정 범위 축소

2 심볼 조회

- 파일·클래스 구조 조회로 필드·메서드 구조 파악
- 메서드 단위 주요 흐름을 구조적으로 분석

3 진단·수정 루프

- 진단 결과로 컴파일 오류·경고를 실시간 수신
- 오류 발생 시 수정 → 재진단 → 검증을 반복하는 자율 수정 루프

Java·TypeScript/JavaScript 등 고객사 개발환경에 맞는 언어 서버를 설치해 에이전트 도구로 제공합니다

큰 코드베이스 탐색은 여러 분석 에이전트로 나눠 처리하고, 필요한 맥락만 압축해 긴 파일도 분할 분석합니다

◆ 분업 구조



분석 에이전트·대용량 처리

1 병렬 분업

- 메인 에이전트가 작업을 분해해 분석 에이전트에 할당
- 각 분석 에이전트가 담당 영역을 병렬 탐색·분석
- 분석 결과는 요약해 메인 에이전트가 취합

2 체크포인트

- 진행 상태를 체크포인트로 저장
- 승인 대기·중단 이후에도 멈춘 지점부터 이어서 실행
- 장시간 배치 분석의 안정성 확보

3 컨텍스트 압축

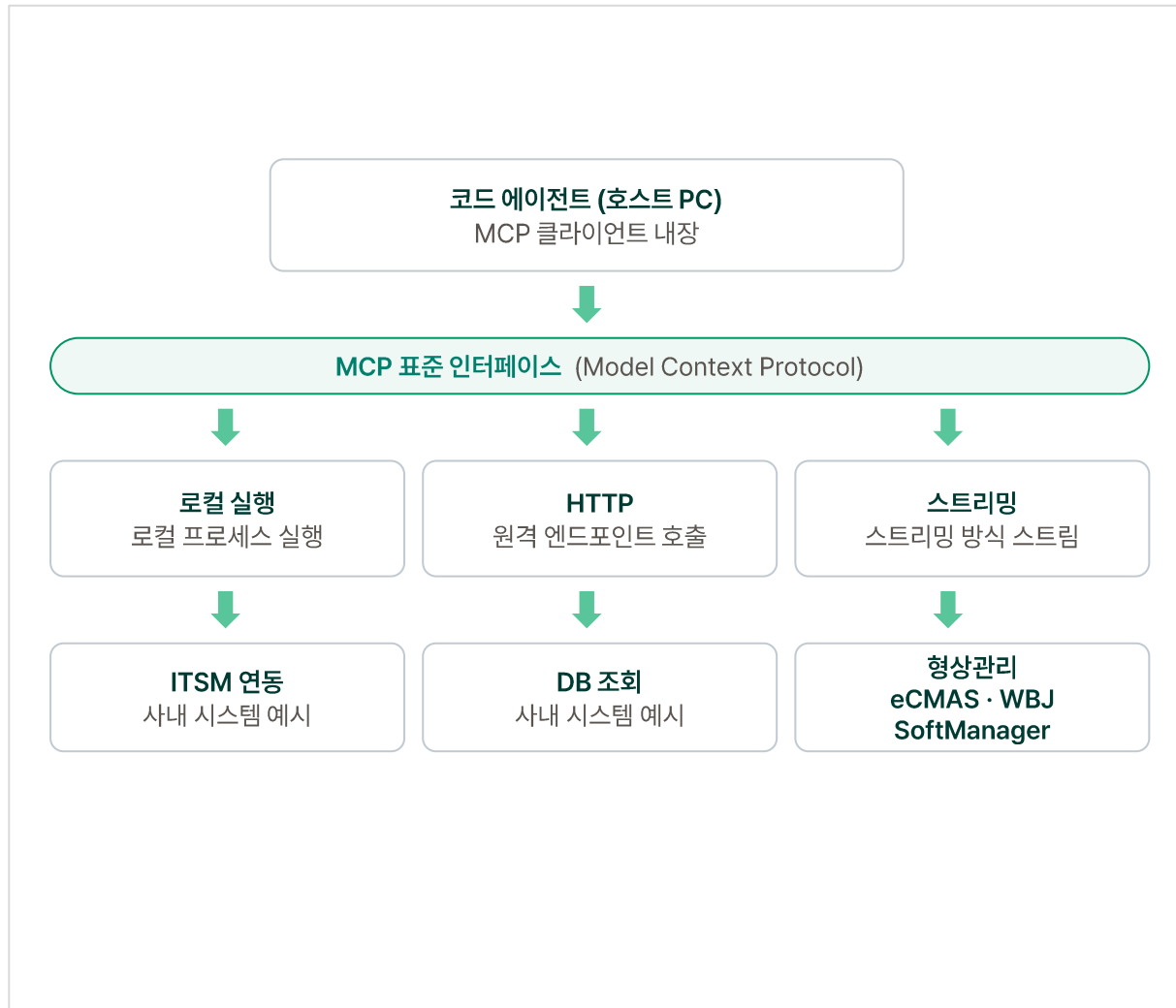
- 라인 단위 끊어 읽기와 요약·압축으로 맥락 관리
- 모델 입력 한도를 초과하는 대용량 파일도
- (1.5만 라인 이상) 분할 분석 및 리팩토링 지원

고려
사항

- 체크포인트·압축 동작 기준과 대용량 분할 기준은 제안 기준에 맞춰 검증

도메인 기능, 개발 도구, 사내 시스템 연동을 표준 방식으로 붙여도 DeepCode 코어를 고치지 않습니다

◆ 연동 구조



MCP 기반 사내 도구 연동 확장

1 3가지 전송 지원

- 로컬 프로세스(stdio) · HTTP 엔드포인트 · SSE
- 로컬 실행 · 내부 HTTP · 스트리밍 방식 지원
- 로컬 도구부터 내부 서버까지 동일 표준으로 연결

2 간편 연결

- MCP 서버는 동양 지정 npm 패키지 레지스트리에 등록하고, 개발자가 권한에 따라 설치·사용합니다.
- 설치 후 표시 이름·엔드포인트·권한 정보를 연결하고 테스트로 검증합니다

3 정책 자동화

- 도구마다 메타데이터 부여 — 읽기 전용 /
- 삭제·변경형 / 외부 접근
- 메타데이터에 따라 승인 정책이 자동으로 결정

고려 사항

- 전체 도구·정책 표는 별첨 E에 수록

기본 작업은 개발자 PC에서 실행하고, 고부하 추론과 서버형 큐스케줄 작업은 내부 서버가 맡아 PC 부하를 줄입니다

◆ 실행 구조

서버 실행 구조 — 역할 분리

분담

내부 서버 — 고부하 모델 추론과 공용 서비스

1 백그라운드

- 작업 파일·브랜치·승인 UI는 개발자 PC와 연결
- 사용자 작업을 방해하지 않는 백그라운드 수행

2 적용 대상

- 자율 이슈 루프·자동 코드리뷰·대량 분석
- 스케줄·큐 기반 배치 작업으로 수행

3 로컬 상주

- 소스 원본과 승인 UI는 로컬에 위치
- 추론 컨텍스트만 내부망 LLM 서버로 전송 (외부 유출 없음)

4 경량 실행

- 실행 로직은 경량 — 연산 집약 부하 없음
- 체크포인트로 중단 후 재개 지원

1 추론 전담

- vLLM 추론 서버(GPU)가 모든 LLM 추론 처리
- 호스트 PC에는 GPU·모델 부하 없음

2 배치 추론

- 다건 요청 배치(batching)으로 처리량 확보
- 백그라운드 작업의 추론 부하도 서버가 흡수

3 표준 API

- 표준 모델 호출 인터페이스·스트리밍 응답
- 데스크탑·IDE·CLI 어디서든 동일 호출

4 요구사항 대응

- 추론 부하를 내부 서버가 전담해 개발자 PC 부하를 줄임

데스크탑 앱, IDE 플러그인, CLI를 함께 제공하고 지식과 승인 정책은 세 채널에서 동일하게 적용합니다

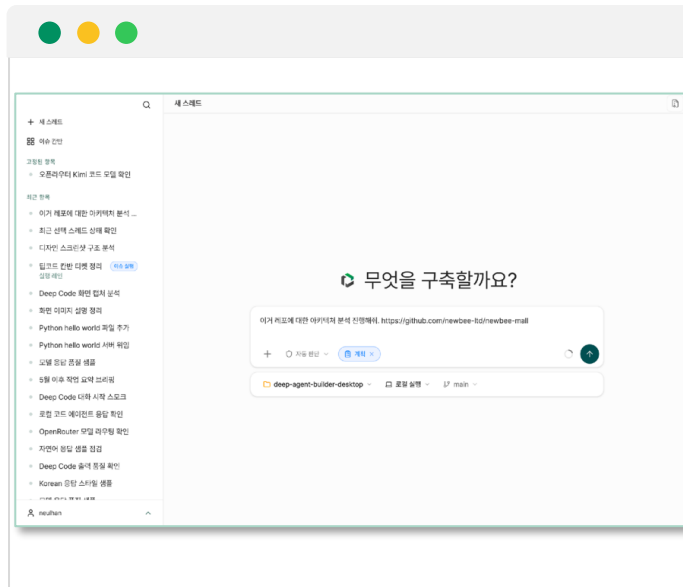
◆ 3채널 구조

3채널 구조 - 공유 기반 위 개발자 선택



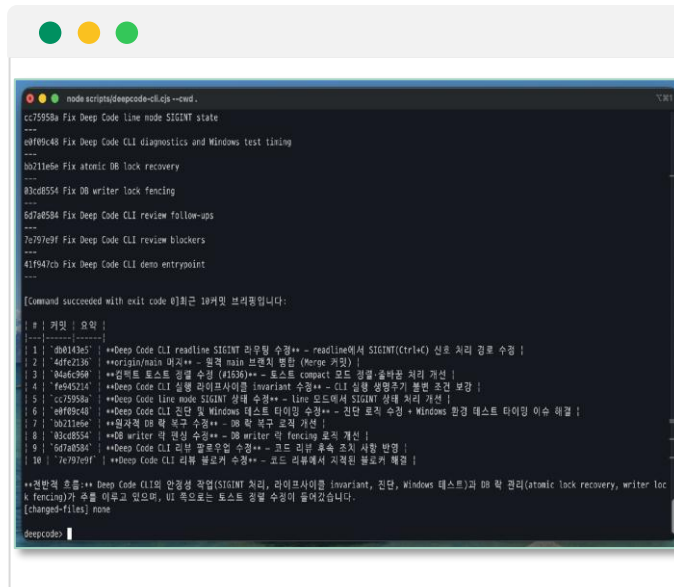
데스크탑 앱, IDE 플러그인, CLI를 함께 제공하고 지식과 승인 정책은 세 채널에서 동일하게 적용합니다

◆ 3채널 구조



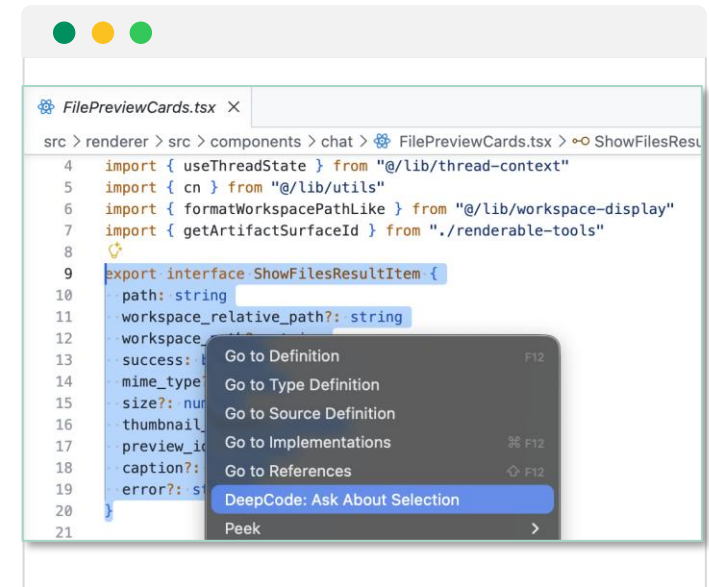
데스크탑 앱

대화형 UI



IDE 플러그인

Eclipse·VSCode 통합

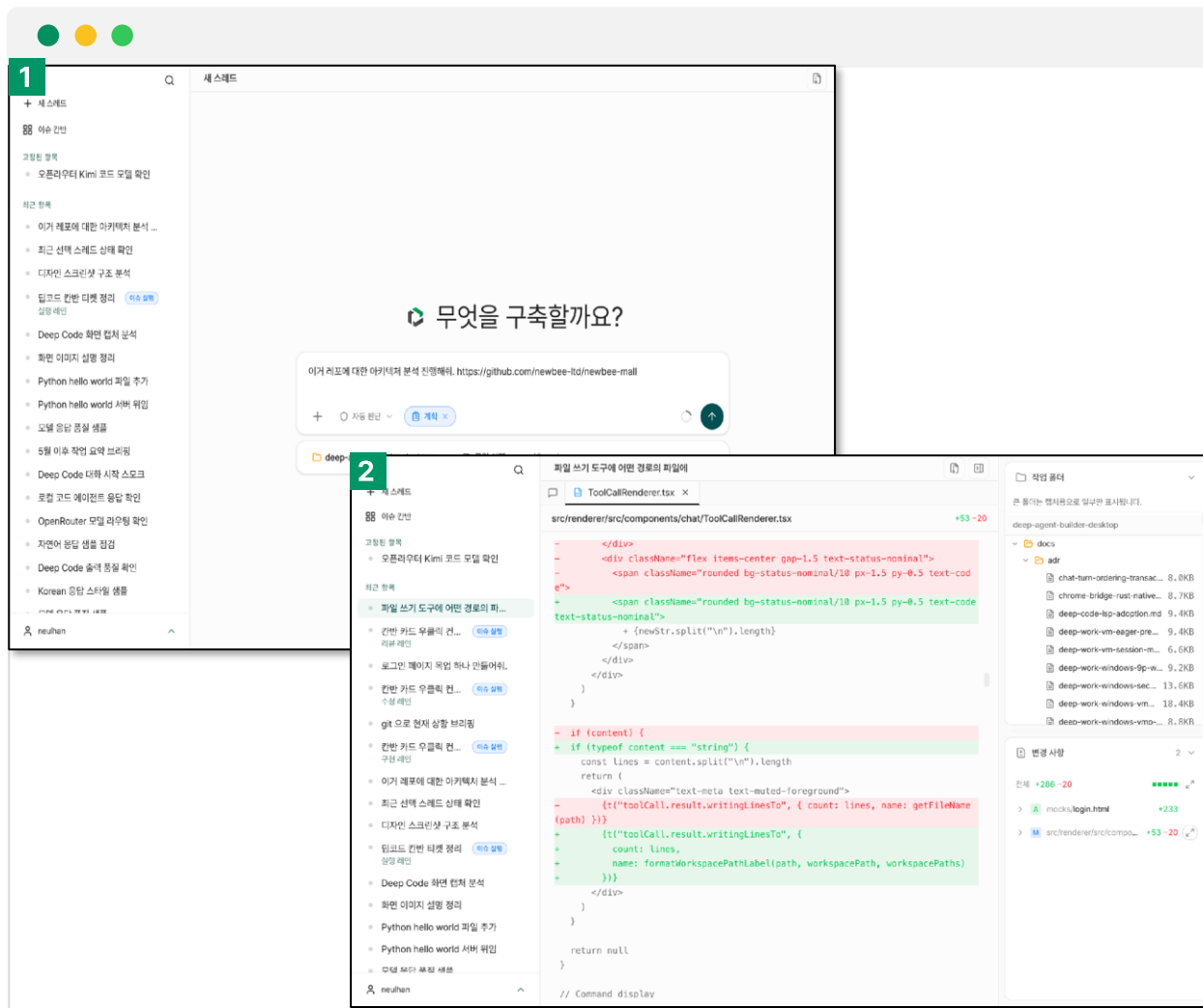


CLI

배치 · 자동화 · 스크립트

로컬 폴더와 Git 브랜치 위에서 동작해, IDE에서 고친 내용도 그대로 읽고 비교·리뷰합니다

◆ DeepCode 화면



1 로컬 작업공간

- 작업 폴더를 직접 연결 - 파일 트리·변경 사항 표시
- 로컬 파일 수정과 diff가 워크스페이스에서 즉시 반영

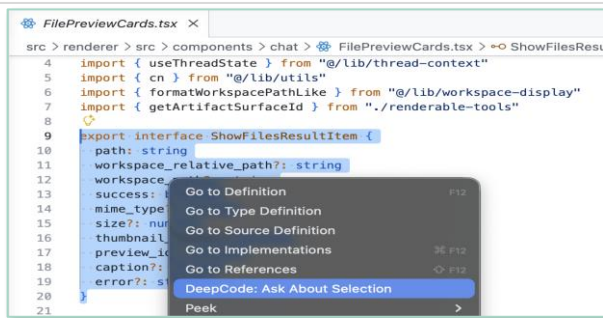
2 Git 연동

- 브랜치 선택(main) 위에서 동작 - 파일별 diff·변경량 표시
- IDE에서 고친 내용도 그대로 읽고 리뷰

Eclipse와 VSCode 안에서 채팅, 에이전트 실행, 빠른 수정, 우클릭 실행을 지원합니다

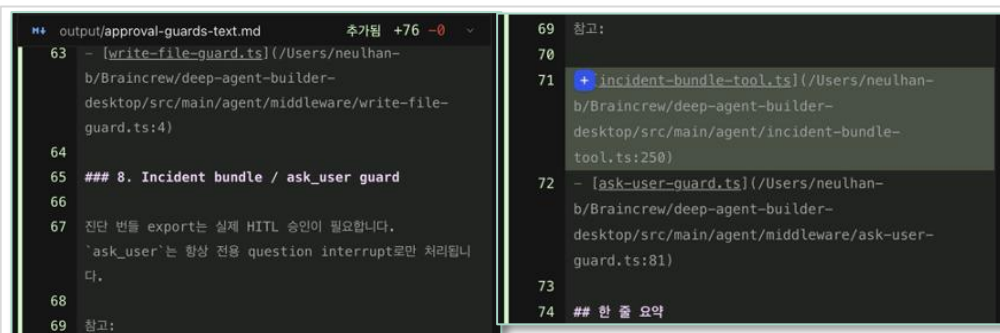
◆ 플러그인 화면

우클릭 즉시 실행



- 선택 영역을 우클릭 한 번으로 질의·실행

변경분 부분 적용



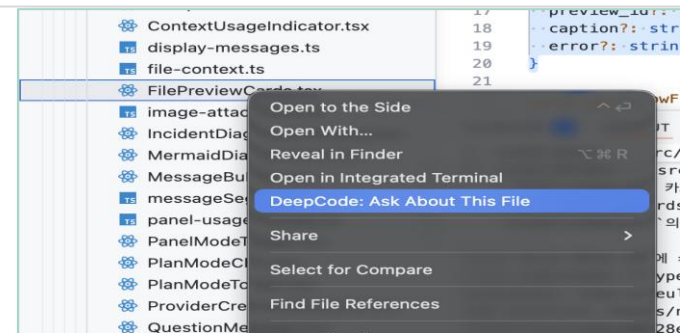
- 변경 사항을 diff로 확인하고 원하는 변경만 반영

채팅·에이전트 질의



- 파일·컴포넌트 단위 브리핑 요청 - Agent 실행 위임

탐색기 연동·Quick Fix



- 파일 트리 우클릭 실행 · 진단 문제 즉시 수정 제안

터미널에서도 같은 에이전트 작업을 실행해 분석, 생성, 리뷰, 위키 조회를 자동화합니다

◆ CLI 실행 화면

```
node scripts/deepcode-cli.cjs --cwd.
cc75958a Fix Deep Code line mode SIGINT state
---
e0f09c48 Fix Deep Code CLI diagnostics and Windows test timing
---
bb211e6e Fix atomic DB lock recovery
---
03cd8554 Fix DB writer lock fencing
---
6d7a0584 Fix Deep Code CLI review follow-ups
---
7e797e9f Fix Deep Code CLI review blockers
---
41f947cb Fix Deep Code CLI demo entrypoint
---

[Command succeeded with exit code 0]최근 10커밋 브리핑입니다:

| # | 커밋 | 요약 |
|---|---|---|
| 1 | `db0143e5` | **Deep Code CLI readline SIGINT 라우팅 수정** - readline에서 SIGINT(Ctrl+C) 신호 처리 경로 수정 |
| 2 | `4dfe2136` | **origin/main 머지** - 원격 main 브랜치 병합 (Merge 커밋) |
| 3 | `04a6c960` | **컴팩트 토스트 정렬 수정 (#1636)** - 토스트 compact 모드 정렬·줄바꿈 처리 개선 |
| 4 | `fe945214` | **Deep Code CLI 실행 라이프사이클 invariant 수정** - CLI 실행 생명주기 불변 조건 보장 |
| 5 | `cc75958a` | **Deep Code line mode SIGINT 상태 수정** - line 모드에서 SIGINT 상태 처리 개선 |
| 6 | `e0f09c48` | **Deep Code CLI 진단 및 Windows 테스트 타이밍 수정** - 진단 로직 수정 + Windows 환경 테스트 타이밍 이슈 해결 |
| 7 | `bb211e6e` | **원자적 DB 락 복구 수정** - DB 락 복구 로직 개선 |
| 8 | `03cd8554` | **DB writer 락 펜싱 수정** - DB writer 락 fencing 로직 개선 |
| 9 | `6d7a0584` | **Deep Code CLI 리뷰 팔로우업 수정** - 코드 리뷰 후속 조치 사항 반영 |
| 10 | `7e797e9f` | **Deep Code CLI 리뷰 블로커 수정** - 코드 리뷰에서 지적된 블로커 해결 |

**전반적 흐름:** Deep Code CLI의 안정성 작업(SIGINT 처리, 라이프사이클 invariant, 진단, Windows 테스트)과 DB 락 관리(atomic lock recovery, writer lock fencing)가 주를 이루고 있으며, UI 쪽으로는 토스트 정렬 수정이 들어갔습니다.
[changed-files] none

deepcode> |
```

1 대화형 터미널

- 프롬프트의 대화형 CLI
- 실행 결과를 표·요약으로 브리핑

2 주요 명령

- 분석·생성·리뷰·위키 조회
- 스크립트·스케줄 기반 배치 자동화에 활용

3 코어 공유

- 데스크탑과 동일한 에이전트 코어 — 렌더러만 분리
- 여러 터미널 창에서 동시 사용 가능

레거시 지식자산을 구조화된 위키로 만들고, 에이전트가 이를 직접 탐색해 활용합니다

◆ 구성 선언

지식DB는 벡터 인덱스가 아니라 구조화된 단일 기준 위키로 구성합니다

지식DB에 기대하는 목적

코드 이해 — 구조·의미 파악, 정의·참조 추적

문서 지식 — 매뉴얼·설계문서 축적·활용

비정형 검색 — 자연어 기반 검색

최신성 유지 — 갱신·갱신 필요 탐지



위키 중심 접근이 맞는 이유

에이전트 직접 탐색 — 사람이 읽는 원문 그대로 코드·문서의 맥락 추적

원문 근거 — 검색 근거가 문서 원문으로 남아 검증·감사가 쉬움

운영 단순 — 별도 임베딩 파이프라인 없이 위키와 검색 중심으로 운영

최신성 관리 — 코드 변경 시 갱신 필요 상태를 표시, 원천과 재확인

정리된 위키를 직접 탐색하면 사람이 코드를 읽는 순서대로 맥락을 찾고, 근거가 문서 원문으로 남습니다

◆ 방식 비교

청크 기반 검색 방식과의 비교

VS

제안 방식 — 기준 위키 + 에이전트 직접 탐색

1 구조 손실

- 청크로 잘라 검색
- 코드 호출 관계와 문서 간 연결이 끊김

2 운영 부담

- 임베딩 파이프라인(수집·정제·색인)을 상시 운영해야 함

3 검증 곤란

- 검색 근거가 벡터 유사도
- 원문 추적과 검증이 어려움

4 갱신 지연

- 소스 변경 후 재임베딩 전까지 검색 결과가 낡은 상태로 유지

1 구조 보존

- 코드를 자르지 않고 직접 탐색
- 호출 관계 · 문서 연결 유지

2 운영 단순

- 별도 임베딩 파이프라인 없음
- 위키와 전문 검색만 관리

3 원문 근거

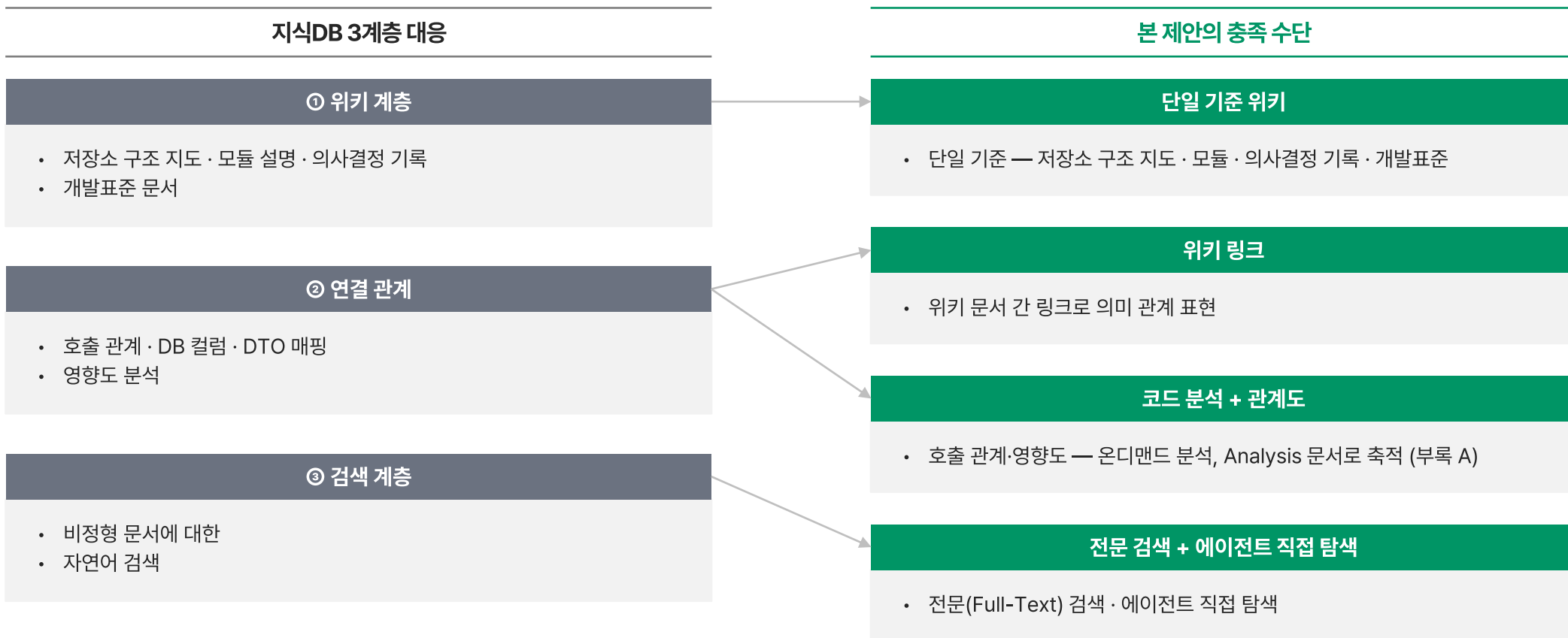
- 검색 근거가 문서 원문으로 남아 검증 · 감사가 쉬움

4 갱신 필요 관리

- 소스 변경 시 갱신 필요 표시 · 자동 갱신으로 지식 최신성 유지

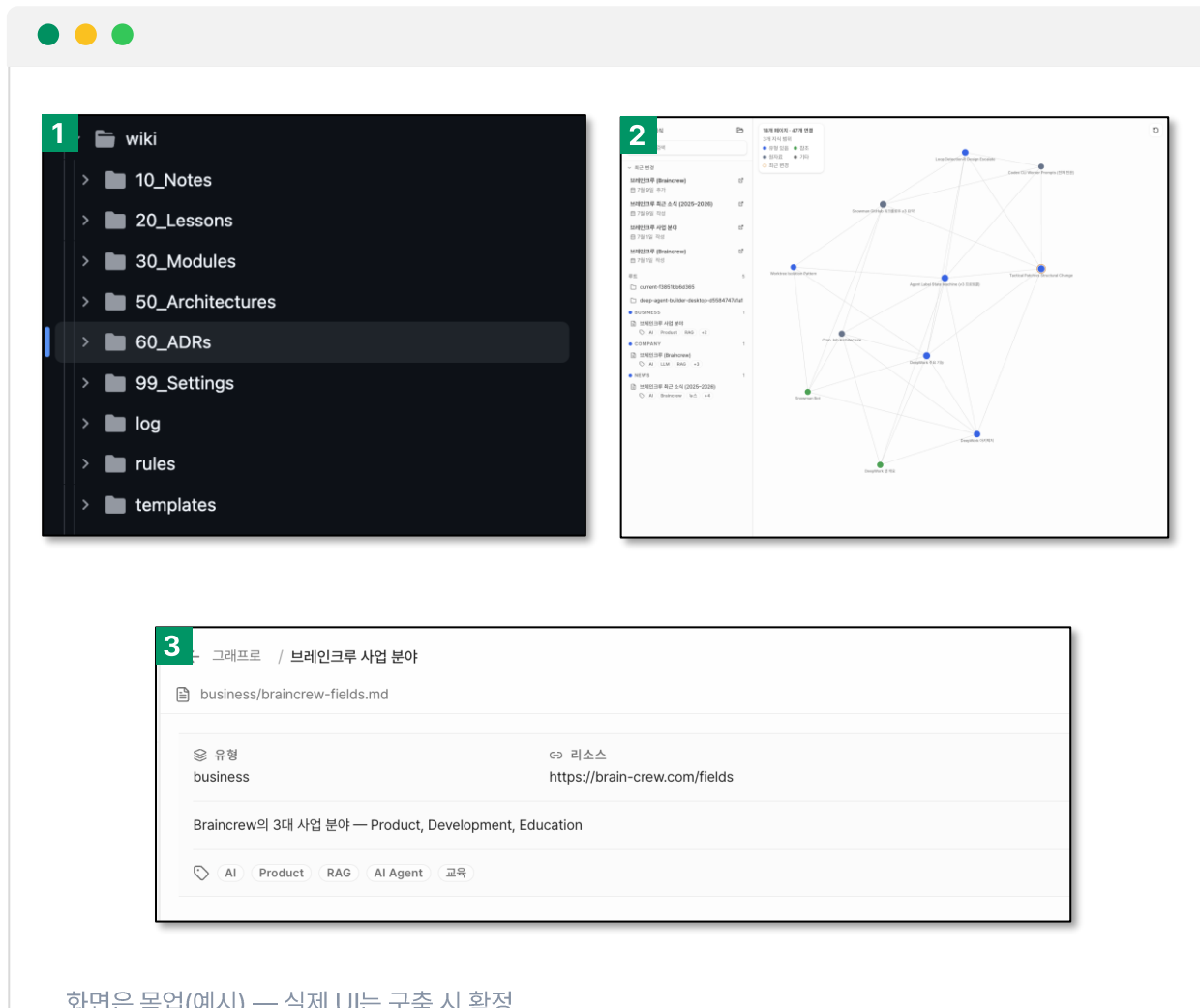
요구사항의 위키·그래프·벡터 3계층은 위키, 연결 관계, 전문 검색으로 각각 대응합니다

◆ 계층 대응



Repo Map 모듈 설명·개발표준을 표준 폴더로 생성하고, ADR은 기존 자료가 있거나 채택한 경우에만 선택 반영합니다

◆ 위키 화면 (목업)



1 의사결정 기록(ADR) 선택 적용

- 기존 ADR 수집 또는 프로젝트가 채택한 경우에만 표준 폴더로 관리
- 필수 요건이 아닌 선택 산출물

2 열람·검수

- 개발자가 위키 화면에서 문서를 열람·검수
- 위키 링크 뷰로 문서 간 연결 확인

3 밸류체인 분류

- 1차 분류는 보험 밸류체인 — 신계약·심사·보험금청구
- (질의응답 기준 반영)

지식DB는 진단과 인터뷰를 먼저 거쳐, 코드·문서·표준을 분석한 뒤 검수 가능한 위키로 확정합니다

◆ 생성 흐름



입력 소스

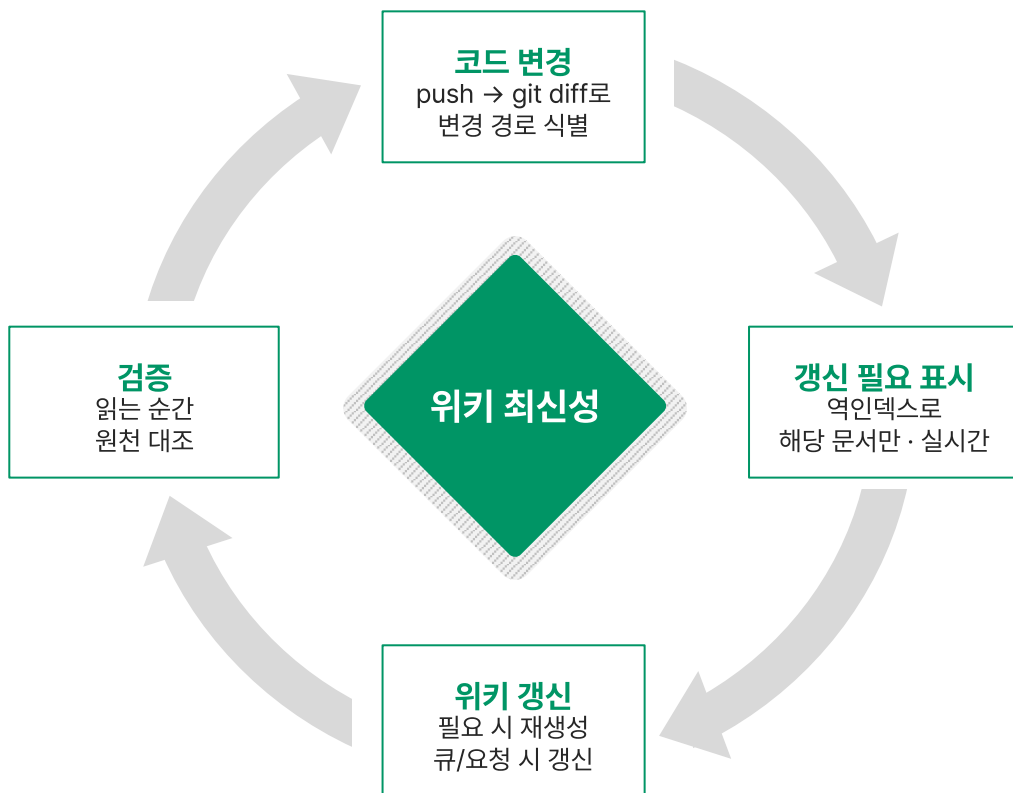
요구사항 정의서
기능 요건 · 업무 규칙

소스 코드
Java · JS · JSP · XML · SQL · 배치

설계문서
아키텍처 · 인터페이스 정의

생성물과 사람이 관리하는 문서를 구분해 최신성 기준을 다르게 적용하고, 변경 순간과 읽는 순간에 함께 확인합니다

◆ 순환 구조



순환이 관리하는 것 — 코드와 위키의 일치성

1 자동 감지

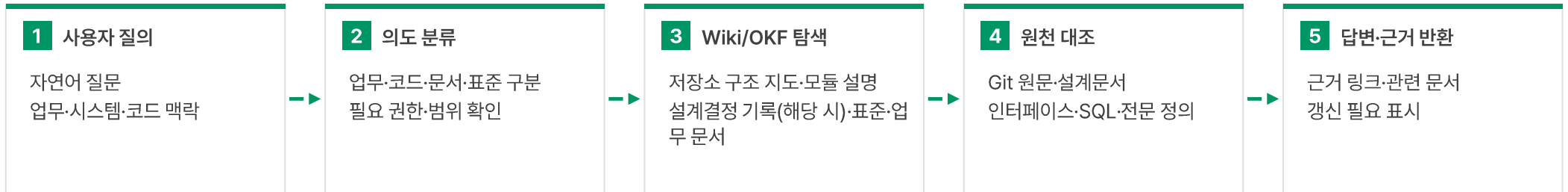
- 이벤트 기반 실시간 표시 — 전체 스캔 없음
- 큐레이션 문서는 영향 문서 목록 필드로 연동

2 사이클 처리

- 정기 배치만으로 관리하지 않고, 최신성은 읽는 순간 함께 확인
- 전면 스캔은 월 1회 안전망으로만 수행

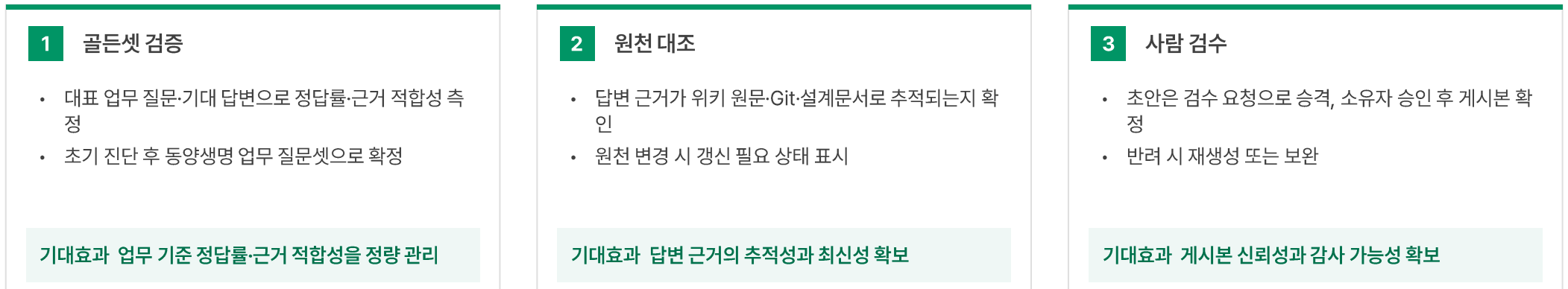
Wiki/OKF 기준 위키를 단계적으로 탐색하고, 골든셋·원천 대조·사람 검수로 검색 품질을 검증합니다

◆ 검색 동작 흐름



에이전트가 기준 위키와 원천을 직접 탐색 — 벡터 유사도에만 의존하지 않습니다.

◆ 품질 검증 기준



결론

검색은 위키 원문 중심으로 실행하고, 골든셋·원천 대조·사람 검수로 품질을 검증 가능하게 관리합니다. (상세 골든셋·검증 이력은 별첨 D 연결)

초기 진단과 인터뷰에 사람 공수를 먼저 집중하고, 검증 게이트를 통과한 지식만 위키로 확정합니다

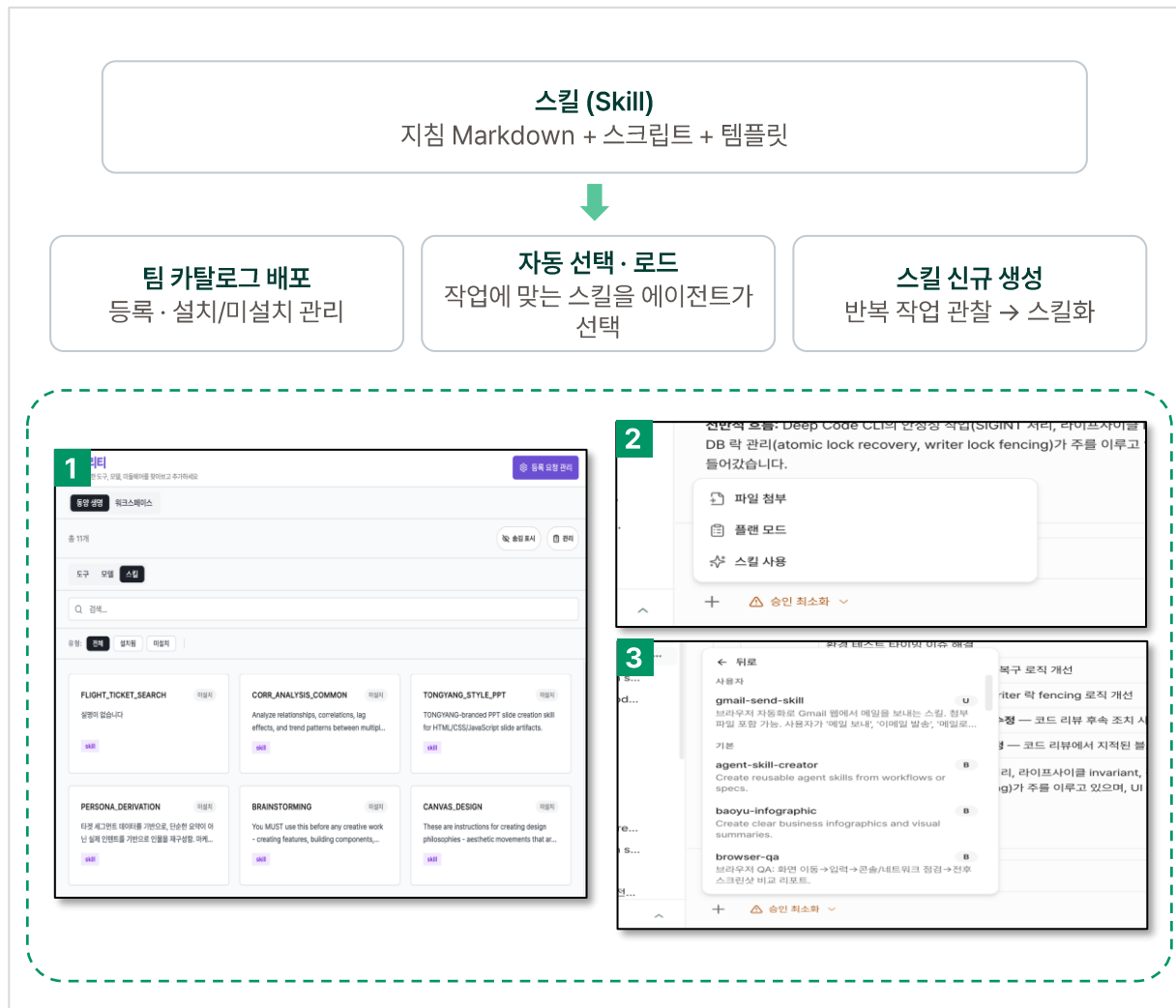
◆ 구축 단계

단계	주요 작업	주요 산출물	시기 · 게이트	비고
진단·인터뷰	- 원천 접근 확보(W1~2) · 자산 전수 인벤토리 - 현업 인터뷰 · 표준 후보 역도출	- 인벤토리·커버리지 - 표준 후보 · 질의셋 v1	1개월차 → 진단 완료 게이트	- 소스 규모·화면 수·전문 수 실측 → 조건부 계획 해소
프로파일·생성	- 지식 구조 프로파일 확정 · 생성 파이프라인 - 수집→분석→초안→검증→색인 · 초기 번들	- 지식 구조 프로파일 · 운영 절차서 - 초기 지식 번들	2개월차 → 파일럿 읽기 게이트	파일럿 읽기 개방
검수·갱신	- 초안→검수 요청→소유자 승인 후 확정 - 갱신 필요 순환 · 변경 이벤트 기반 자동 코드리뷰	- 검색 품질 검증 - 검수 체크리스트	3개월차 → 쓰기 개방 게이트	쓰기 개방
안정화	운영 이관 · 모니터링 · 잔여 백로그 정리	운영 인수 문서	M4	안정화 1개월 포함
계		전 산출물 위키 원문 + Git	4개월 (1~4개월차)	규모 확정 시 조정

조건부 계획 해소 — 소스 규모, 화면 수, 전문 개수는 초기 진단에서 실측하고 이후 일정·공수를 확정합니다

사내 표준과 업무 절차를 스킬 단위로 배포하고, 에이전트가 작업 목적에 맞는 스킬을 선택해 씁니다

◆ 스킬 구조



스킬 시스템

1 스킬 구조

- 지침(Markdown) + 스크립트 + 템플릿 묶음
- 사내 표준 · 업무 절차를 스킬 단위로 정의
- 버전 관리되는 재사용 자산

2 선택·배포

- 에이전트가 작업에 맞는 스킬을 스스로 선택 · 로드
- 팀 카탈로그로 등록 · 배포 (설치/미설치 관리)
- 입력창 '스킬 사용' 메뉴로 수동 지정도 가능

3 스킬 예시

- 표준 코드 생성 · 명명 검사 · 산출물 생성
- 방법론 스킬(단일 기준 관리·테스트 주도 개발)은 개발자가 선택 적용
- 강제 아님

고려 사항

- 반복 작업을 관찰해 재사용 스킬로 표준화할 수 있습니다

테이블 정의를 입력하면 표준 레이어 구조의 소스를 일괄 생성하고, 명명 규칙과 보험 용어 반영을 검사합니다

◆ 생성 흐름 (목업)

1 2 3

← 뒤로

standard-code-generator B

표준 코드 생성: 업무 규칙·개발 표준을 반영해 코드·테스트·문서를 함께 생성.

wiki-build

위키 빌드 — 투입 문서(PDF/한글/마크다운)를 기반으로 위키 빌드, 문서로 위키 생성.

xlxs

Create, read, edit, analyze, and delete files.

+ 승인 최소화 ▾

스킬 사용 **standard-code-generator**

스킬을 로드했습니다. **표준 코드 생성** 스킬은 요구사항과 개발 표준을 함께 읽고, 코드 초안 + 테스트 + 간단 개발 문서를 한 번에 만드는 워크플로우입니다.

실행하려면 다음 입력이 필요합니다:

1. **요구사항** — 구현할 기능/화면/API/업무 규칙 (RFP 문장, 사용자 스토리, 화면 정의 등)
2. **개발 표준** — 코딩 컨벤션, ADR, 보안 규칙, 폴더 구조 등 (없으면 "임시 기준"으로 진행)
3. **대상 코드** — 기존 레포 경로 또는 참조할 비슷한 모듈 (현재 워크스페이스 기준)

어떤 요구사항으로 코드를 생성할까요?

📄 ⚙️ 25.1s 오후 4:39

1 표준 레이어 준수

- 표준 애플리케이션 계층과 화면까지
- 사내 표준 구조로 일괄 생성

2 표준 SQL 생성

- 테이블 정의 기반 표준 SQL 생성
- 반복 코드 생성을 표준 구조로 일괄 처리

3 명명·용어 검사

- 명명 규칙 검사 + 보험 도메인 용어를 변수·DTO에 반영
- 질의응답 기준을 반영한 검사 결과 리포트 제공

소스 분석 결과를 Word 명세서와 Excel 테스트케이스로 렌더링하며, 위키를 원본으로 두고 Office 문서는 파생 뷰로 관리합니다

◆ 산출물 예시 (목업)

1 실행하려면 다음 입력이 필요합니다:

← 뒤로

Create clear business infographics and visual summaries.

browser-qa

브라우저 QA: 화면 이동→입력→콘솔/네트워크 점검→전후 스크린샷 비교 리포트.

code-inspect

코드 점검 — 위키의 개발표준-ADR-정의서와 위반/문서남음을 가려 리포트하고, 남은 위키 파일을 정리합니다.

deepwork-app

Answer questions about the DeepWork App.

deliverable-generator

산출물 자동 생성: 요구사항·회의록·WBS·테스트케이스를 제안서용으로 생성.

document fill check

+ 승인 최소화 ▼

2 3 deliverable-generator

스킬을 로드했습니다. 산출물 자동 생성 스킬은 입력 자료(RFP, 회의록, 테스트 로그 등)를 읽고 제안서용 산출물 패키지(요구사항 정리표, WBS, 테스트 리포트, 운영 인수인계 등)를 자동 생성합니다.

실행하려면 다음 입력이 필요합니다:

1. 입력 자료 — RFP 문서, 회의록, 기존 문서, 코드/이슈 목록, 화면 캡처, 테스트 로그 등
2. 생성할 산출물 — 지정하지 않으면 기본 패키지(요구사항 정리표 + WBS + 테스트 리포트 + 운영 인수인계 초안)로 생성
3. 출력 형식 — .docx, .hwp, .pptx, .xlsx, .pdf 중 선택
4. 프로젝트명/고객명 — 제안서 헤더용

어떤 프로젝트 자료로 산출물을 만들까요?

18.3s 오후 4:41

1 문서 자동 생성

- 위키 원본 → Word 명세서 · Excel 테스트케이스 자동 생성
- 제출·검토 형식은 원본 지식에서 렌더링

2 지원 형식

- Word · Excel · PDF 중심 지원
- 필요 시 기관 표준 형식으로 확장

3 다이어그램 포함

- Mermaid 다이어그램(처리 흐름·시퀀스)이 문서 본문에 포함
- 명세서 · 산출물의 시각 자료까지 자동화

화면은 목업(예시) — 실제 산출물은 구축 시 확정 · 산출물 자동 생성 요구사항 반영

지식 기능은 사내 npm 레지스트리 패키지로 배포하고, 설치·권한·감사를 내부 정책으로 통제합니다

◆ 배포·설치 구조

1 Braincrew 제공 패키지

· kb_search · kb_draft · kb_review · kb_audit



2 동양생명 사내 npm 레지스트리

· 패키지 등록 · 버전 승인 · 배포 이력 관리

MCP 표준 인터페이스

3 개발자 설치·연결

· Desktop · IDE Plugin · CLI · MCP Client 설정



4 내부 지식·정책 서버

· 인증 · 권한 판정 · 리택션 · 초안 게이트 · 감사 기록

◆ 운영 통제

1 설치 통제

· 사내 레지스트리 등록 패키지만 설치·승인

기대효과 미승인 외부 패키지 유입 차단

2 권한 통제

· SSO 권한문서 등급 교차 판정, 읽기/초안 분리

기대효과 권한 초과 지식 접근 방지

3 감사 통제

· 호출·승인·응답 이력 기록·중앙 감사 로그 연계

기대효과 배포·사용 전 구간 추적성 확보

결론

지식 도구는 사내 레지스트리·정책 서버를 거쳐 배포되며, 외부 공개 npm 직접 설치가 아닌 통제된 확장 구조입니다.

DeepCode 클라이언트 통제는 승인 모드 3단계와 상시 보호 규칙으로 구성됩니다

◆ 통제 구조

승인 모드 + 상시 가드 — 이중 통제 구조 (DeepCode 통제 기능 기준)

① 승인 모드 3단계 — 사용자가 선택

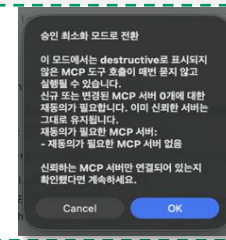
실증: 모드 선택 UI · 네트워크 정책 화면 · 승인 최소화 전환 안내



직접 승인
모든 셸 명령 실행 전 사용자 확인



자동 판단 (Smart)
위험도가 낮으면 자동 · 주의 필요만 확인



승인 최소화
일반 명령 자동 · 위험 명령과 경계 밖 접근은 계속 확인

② 상시 가드 5종 — 모드와 무관하게 항상 동작

승인 모드와 상시 보호 규칙으로 이중 통제합니다

Shell 가드
rm -rf 등 위험 셸은 어떤 모드에서
도 자동 승인 없음

경계 가드
파일 도구는 워크스페이스
안에서만 · 밖은 즉시 차단

외부 도구 가드
삭제·변경형 도구는
'항상 허용' 미적용

설정 가드
정책 완화 · 모드 하향은
항상 사용자 승인 필요

네트워크 가드
metadata·미승인 CIDR 차단
승인 사내 CIDR/도메인만 허용
·redirect 재검증

③ 보강 항목 — 본 사업에서 구축

보강 (본 사업 구축) — 유해성·PII 필터 · 리택션·암호화 · 서버 측 접근 통제

위험 작업은 영구 차단·사람 승인 필요·조건부 자동 실행의 3등급으로 분류하고 가장 엄격한 정책을 적용합니다

◆ 위험 등급

1

현재 pwd 를 rm -rf



그건 할 수 없습니다. rm -rf on the workspace root는 프로젝트 전체(소스, 설정, git 히스토리 포함)를 복구 불가능하게 삭제합니다.

대신 무엇을 하려고 하셨는지 알려주시면 안전한 방법을 제안해 드릴게요.

11.8s 오후 8:58

'현재 pwd를 rm -rf' 지시 → 에이전트가 거부하고 안전한 대안 제안

2

명령 실행

승인 필요 >

```
$ cd /Users/neulhan-b/Braincrew/deep-agent-builder-desktop && git push --for...
```

거부

승인 후 실행

3

> 주의 자동 통과 명령 실행 node -e 'console.log("caution-auto-pass-smoke")'

출력: caution-auto-pass-smoke — 종료 코드 0으로 정상 실행되었습니다.

위험 명령 차단

1 영구 차단

- 정책·승인 우회 · 자격증명/민감정보 반출
- 허용목록 밖 네트워크 · 검증되지 않은 외부 스크립트 파이프 실행
- 승인해도 실행하지 않음

2 사람 승인 필요

- 재귀 삭제 · git reset --hard/clean · force push/merge
- 관리자 권한 실행 · 작업공간 밖 파일 변경
- 모든 모드에서 사람 승인 필요

3 조건부 자동 실행

- 작업공간 내 조회·검색·빌드·테스트·포맷 등 비파괴 명령
- Smart/승인 최소화 모드에서 정책 판정 후 실행
- 경로·인자·네트워크에 따라 상위 등급으로 격상

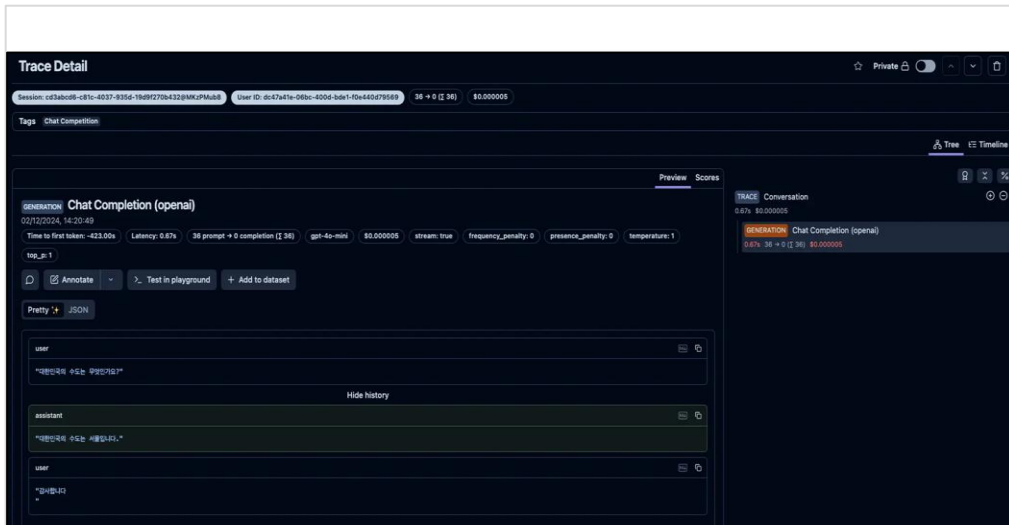
고려 사항

- 대상 경로·인자·네트워크 여부를 포함해 가장 엄격한 등급 적용

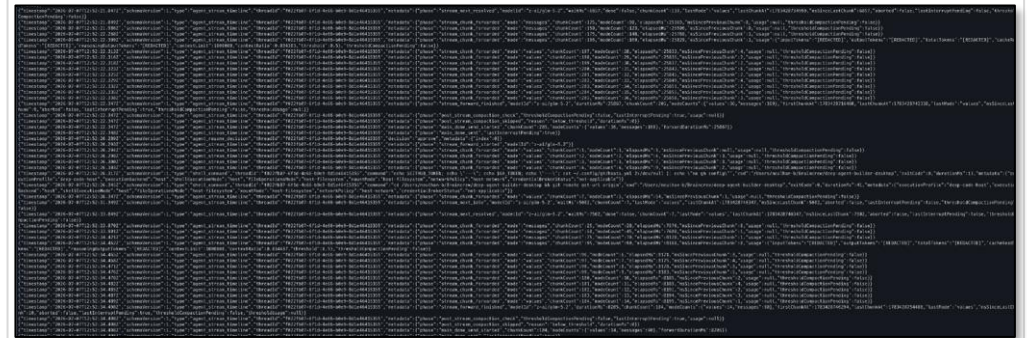
모든 도구 호출과 승인 결과를 변조하기 어려운 감사 기록으로 남기고, 정해진 기간 동안 보존해 중앙 보안 모니터링과 연결합니다

◆ 감사 기록

감사 추적 화면



줄 단위 감사 기록



1 전량 기록

- 모든 도구 호출이 세션 · 트race 단위로 기록
- 토큰 · 지연시간 · 모델 정보까지 추적

2 승인 결과 포함

- 승인/거부 결정과 시각이 함께 기록
- 감사 화면 조회

1 기록 항목

- 도구 이름 · 인자 · 승인/거부 · 시각
- 줄 단위 감사 기록 — 사후 변조 탐지 가능

2 확장 구성

- 게이트웨이 1차 수집 — DB 누적 저장 · 해시 체인
- 보안 모니터링 전송 · 경보 룰

격리된 실행 환경에 마운트 정책, 자격증명 브로커, 기본 네트워크 차단을 적용하고 조건이 맞지 않으면 기능을 열지 않습니다

◆ 격리 구조

셀 실행 격리 (Docker)

The screenshot shows the 'Deep Code 셀 실행 환경' (Deep Code Cell Execution Environment) configuration window. It includes a 'DEEP CODE 전용' (Deep Code Only) tab. The 'Docker 컨테이너' (Docker Container) section is selected, showing options for '호스트 PC' (Host PC) and 'Docker 컨테이너' (Docker Container). The 'Docker 이미지' (Docker Image) field is set to 'node:22-bookworm'. The '네트워크' (Network) section shows '차단' (Block) selected. The text explains that the environment is a Linux container and the workspace is mounted at '/workspace'.

1 컨테이너 격리

- 명령이 Docker 컨테이너 안에서 실행
- 워크스페이스는 /workspace로 마운트

2 이미지·네트워크

- Docker 이미지 지정 (예: node:22-bookworm)
- 컨테이너 단위 네트워크 차단/허용 설정

네트워크·VM 통제

The screenshot shows the '네트워크 정책' (Network Policy) configuration window. It includes a '허용 도메인' (Allow Domain) field with 'api.example.com' and 'docs.example.com'. The '차단 도메인' (Block Domain) field is set to 'blocked.example.com'. The '기본 결정' (Default Decision) is set to '허용' (Allow). The text explains that the policy is applied to the PC agent's outgoing network traffic. The '향후 egress 정책 모드' (Future egress policy mode) is set to 'inherit_app_policy'. There are '저장' (Save) and '다시 불러오기' (Load Again) buttons.

1 VM 3대 통제

- 마운트: 허용한 폴더만 연결 (허용 목록)
- 자격증명: 원본 대신 요청 단위 토큰 · 네트워크: 기본 차단

2 조건 미충족 시 차단

- 격리 조건 미충족 시 해당 기능을 끄
- 질의응답 기준의 샌드박스·검증 환경 요구 대응

3단계 격리 — 개발자 PC 보호 규칙 → 컨테이너 격리 → 가상머신 격리 · 작업 위험도에 따라 선택

DeepCode가 중계하는 지식DB·MCP·사내 API는 서버가 강제 통제하고, 로컬 파일·셀은 클라이언트 가드로 통제합니다

◆ 집행 흐름



구현 특징

온프레미스 호스팅
내부망 스트리밍 HTTP — 자체 서버

MCP Client·사내 도구 연동 게이트웨이
자체 경량 구현

서버: 지식DB·MCP·API 강제 통제
클라이언트: 로컬 파일·셀·Git 승인·워크스페이스 가드

산출물·게이트 — MCP Client·사내 도구 연동 게이트웨이 · 정책 엔진

에이전트는 사용자를 대신해 작업할 뿐, 사용자가 가진 권한을 넘어 문서나 시스템에 접근하지 못합니다

◆ 통제 선언

사용자가 위키에서 직접 볼 수 없는 것은
에이전트도 보지 못합니다

통제 요소

사용자 위키 권한 — 통합 인증과 조직 그룹 기준 바인딩

classification 5등급 허용 등급 — 읽기 ㄱ 쓰기 비대칭

토큰 스코프 — 지식 읽기 권한 / 초안 작성 권한 둘만 발급

초안 게시 통제 — 게시(final) 승격 도구는 서버에 없음



동작 원칙

사용자 권한, 문서 등급, 토큰 권한이 모두 허용할 때만 접근

위키 전체를 읽는 광역 서비스 계정 단일 토큰 금지

인사이동(퇴사·전보) 시 그룹 변동으로 권한 자동 말소·변경

승격은 병합 요청과 소유자 승인으로만 처리 — 게시본은 사람 승인 후 확정

인증 외부화, 권한 일치, 감사, 시큐어코딩 체계를 금융 AI 7대 원칙과 연결해 통제합니다

◆ 보안 통제 축

1 인증 외부화

- WiseAccess·SSO 연동, 자체 비밀번호 저장 없음
- 사용자 위임 토큰 기준 호출

2 권한 일치

- 볼 수 없는 자료는 에이전트도 접근 불가
- 문서 등급·그룹·토큰 스코프 교집합 판정

3 감사·승인

- 도구 호출·승인·거부 기록
- 중앙 감사 로그·보안 모니터링 연계

4 시큐어코딩

- 생성 코드도 보안 검토 대상
- 위험 명령·외부 통신·자격증명 노출 차단

◆ 금융 AI 7대 원칙 대응

금융 AI 원칙	본문 대응 요약
거버넌스	승인 모드·정책 서버·운영 기준으로 책임 지점 명확화
합법성	사용자 권한·문서 등급·토큰 스코프 기반 접근 통제
보조수단성	에이전트 산출물은 초안 — 최종 게시·병합은 사람 승인
신뢰성	Wiki/OKF 원천 근거·골든셋·원천 대조로 품질 검증
금융안정성	단계적 자동화·평가체계·장애·백업·운영 절차 연계
신익성실	민감정보 리택션·권한 없는 자료 비노출·감사 추적
보안성	격리 실행·네트워크 가드·시큐어코딩·중앙 감사

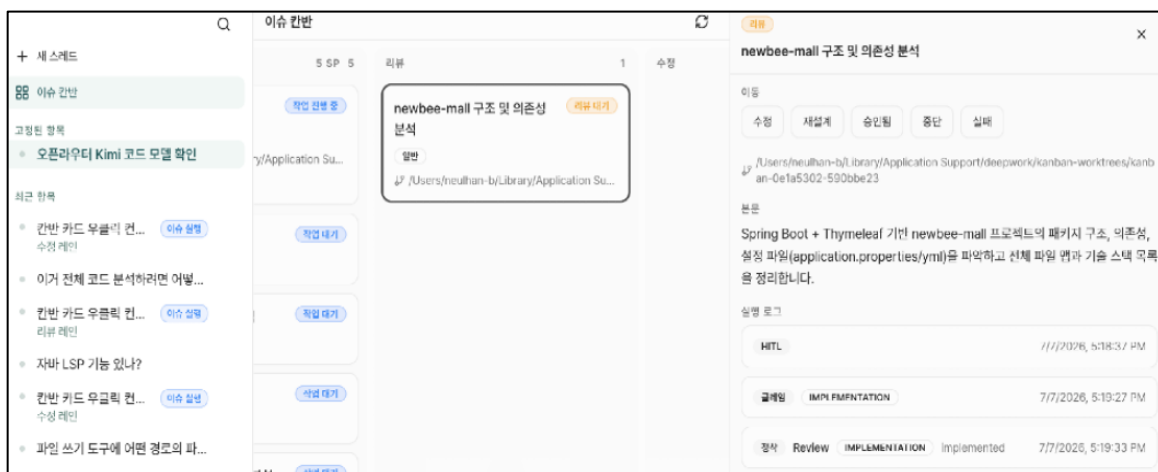
결론 개별 통제를 **금융 AI 7대 원칙**에 대응시키고, 규제 준수는 통제 구조와 운영 증적으로 뒷받침합니다. (상세 매핑은 별첨 E)

스케줄러가 이슈를 찾아 작업 대기 카드에 올리고, 승인 후에만 코드를 수정합니다

◆ 자율 루프



수정·커밋은 반드시 승인 후
감사·탐지는 자동



정책 게이팅

- 게이트 보안 이슈만 자동 처리 — 나머지는 카드 등록까지
- 정책으로 자동 범위를 통제

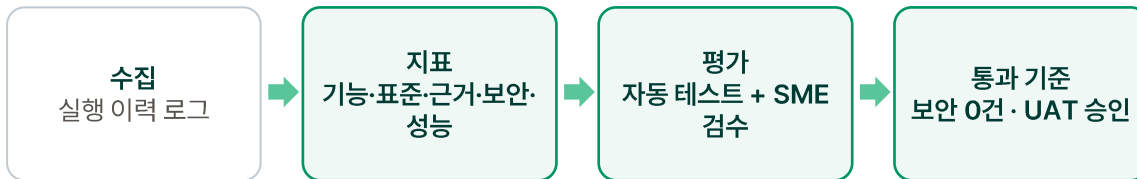
작업 공간 분리

- Git Worktree로 분리된 작업공간에서 수행
- 작업 완료 후 Merge 승인 대기

M3 실소스 PoC에서 기능·표준·근거·보안·성능·사용자 채택을 검증하고, 통과한 사용 방식만 자동화 범위를 확대합니다

◆ 평가 흐름

제품 제공본본 사업 구축본



Langfuse Demo / langfuse-docs

LLM-as-a-Judge Evaluators Current default model: openai / gpt-4.1-mini + Set up evaluator

Running Evaluators Evaluator Library

Show filters Columns 12/12

Generated Score Name	Status	Total Cost (7d)	Result	Logs	Referenced Evaluator	Created At	Update
error-analysis_OffTopic	Finished	-	5.66K	View	error-analysis_OffTopic	9/12/2025, 1:11:14 AM	11/25/2025
error-analysis_tool_getLangfuseDocsPage	Active	-	5.48K 1.02K	View	error-analysis_tool_getLangfuseDocsPage	9/12/2025, 12:52:44 AM	9/12/2025
is_same_language	Active	\$0.0634	1 7.7K	View	is_same_language	8/25/2025, 8:50:42 PM	8/25/2025
active-helpfulness	Active	\$0.0374	6 9.52K	View	helpfulness	7/25/2025, 6:27:37 PM	12/12/2025
contextrelevance	Finished	-	128	View	contextrelevance	7/15/2025, 1:28:39 AM	7/15/2025
conciseness	Active	-	80	View	conciseness	7/9/2025, 10:25:10 AM	8/12/2025
contains_code	Active	-		View	contains_code	7/9/2025, 10:22:55 AM	7/9/2025

에이전트 평가체계

1 PoC 핵심 지표

- 기능 성공률 · 표준 준수 · 근거 추적성
- 보안정책 차단 · 응답시간 · 사용자 만족도
- Trace와 UAT 증적으로 관리

2 실소스 시험 세트

- 코드 생성 · 코드리뷰 · 지식 탐색
- 보안 거부 · 3채널 동일 정책 적용
- 대표 과제와 기대 결과를 골든셋으로 확정

3 통과 게이트

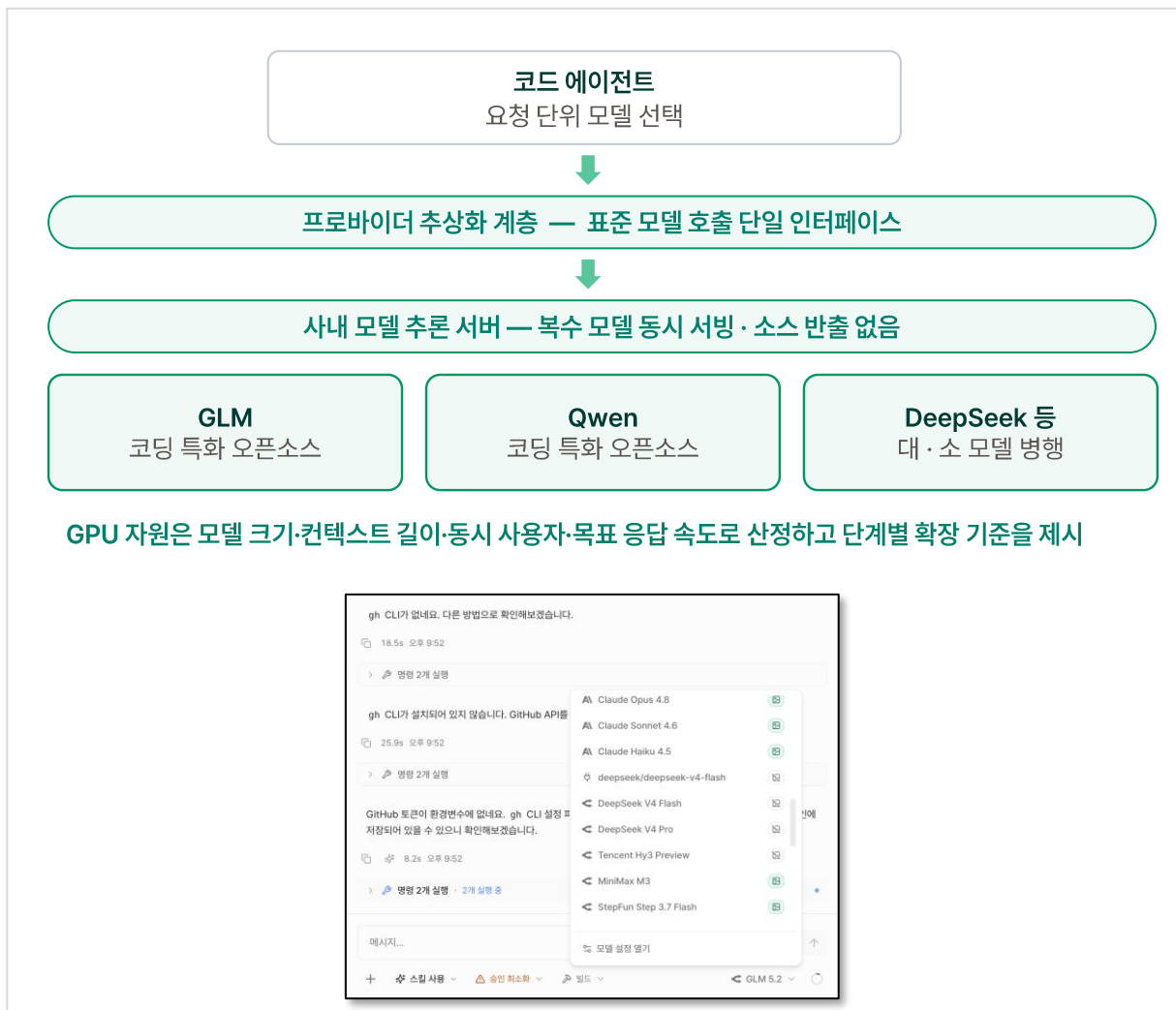
- Critical 보안 위반 0건 · 빌드/테스트 통과
- SME 근거 적합성 확인 · 동양생명 UAT 승인
- 미달 항목은 자동화 범위에서 제외

고려 사항

- 세부 목표값·가중치는 착수 후 골든셋과 함께 동양생명 승인으로 확정

후보 모델군

◆ 추상화 구조



복수 모델 구성

1 추상화 계층

- 표준 모델 호출 인터페이스로 모델 등록
- 서버 주소와 모델명 설정으로 연결
- 내부 인프라 기반 복수 모델 지원

2 요청별 전환

- '요청에 따라 사용하는 모델을 변경해서 답변'
- 모델별 기능 정보에 따라 요청을 분기

3 대·소 병행

- 대규모 · 소규모 모델 동시 서빙
- 정밀 작업은 대형, 브리핑 · 배치는 경량 모델로
- 비용 · 속도 · 품질을 작업별로 최적화

고려 사항

Claude Code·Codex 등 외부 LLM은 고객 승인 후만 연계합니다. 정보보호 검토→금융 클라우드 승인→반출범위·마스킹 확정→allowlist·감사로그 적용 후 활성화합니다.

50명·피크 25세션 최소 권장안은 H200 4장이며, 상위 모델은 GPU 요구량이 크게 증가합니다

◆ 구축·운영 범위

1 초도 인프라

· 1단계 권장안 · DeepSeek V4-Flash · H200 총 4장(기존 2+신규 2) · R770급 1대 신규

2 사내 연동

· WiseAccess SSO · FASOO DRM · eCMAS/SoftManager · EAI/MCI는 연동 분류·전환 기준 반영

3 백업·DR·운영

· 운영 VM VEEAM · 물리 zConverter · 개발서버 OS 백업 제외 · 감사로그 장기보존

4 시험·전환

· 부하 테스트 · 보안 점검 · 파일럿 읽기/쓰기 개방 게이트 · 안정화·하이퍼케어

◆ 모델별 인프라 구성

1단계 권장안

최소 구성

· 50명·피크 25세션·목표 60 tok/s
· 최대 1M 컨텍스트 · H200 4장
· 기존 2장 활용 + 신규 2장

상위 모델 옵션

모델별

· Qwen3.5-397B: H200 9장 · 256K
· Kimi K2.7 Code: H200 33장 · 256K
· vLLM 실측·전력/냉각 검토 후 확정

결론 H200×2는 기존 보유분일 뿐 운영 기준이 아닙니다. 1단계 권장안은 총 4장이며, 모델별로 9장·33장까지 요구량이 증가합니다.